Report for assignment 3

– Godot Code Coverage –

Josefine Nyholm
Albin Blomqvist
Chow Pun Chun
Erik Olsson
Avid Fayaz

Royal Institute of Technology (KTH)
DD2480 VT26
2026-02-18

Contents

Project

Name: Godot
URL: https://github.com/godotengine/godot
Assignment URL: https://github.com/Soffan-Group-6/godot

A cross platform game engine with 2d and 3d capability.

Onboarding experience

Did it build and run as documented?
Yes, but the setup for Linux and Windows was very different, especially since the implementation in Windows was highly reliant on Visual Studio and its provided tools [1]. To be able to compile an additional tool called SCons was required, which was clearly documented. While both setup methods were different in terms of tools and requirements, the build and tests turned out to be successful.

Complexity

1. **What are your results for five complex functions?**
   The initial results produced by Lizard are shown in Figure 1 and Figure 2. It was discovered that manual calculation of the amount of cyclomatic complexity (CC) for the top five functions would not be realistic, although the values provide a necessary basis for further discussion.

```
NLOC  CC Function                                                                    File
----  -- --------                                                                    ----
3336 107 "BaseMaterial3D::_update_shader@686-4128@.\scene\resources\material.cpp"                ".\scene\resources\...
1612 526 "ShaderLanguage::_parse_expression@6008-7887@.\servers\rendering\shader_language.cpp"   ".\servers\renderin...
1578 489 "ShaderLanguage::_parse_shader@9219-11139@.\servers\rendering\shader_language.cpp"      ".\servers\renderin...
1209  19 "GDScriptParser::parse_enum@1575-2796@.\modules\gdscript\gdscript_parser.cpp"           ".\modules\gdscript...
1150 369 "DisplayServerWindows::WndProc@5186-6577@.\platform\windows\display_server_windows.cpp" ".\platform\windows...
```

Figure 1: Top five longest C++ functions

```
NLOC  CC Function                                                                    File
----  -- --------                                                                    ----
1612 526 "ShaderLanguage::_parse_expression@6008-7887@.\servers\rendering\shader_language.cpp"   ".\servers\renderin...
1578 489 "ShaderLanguage::_parse_shader@9219-11139@.\servers\rendering\shader_language.cpp"      ".\servers\renderin...
1080 385 "TextEdit::_notification@737-2039@.\scene\gui\text_edit.cpp"                           ".\scene\gui\text_e...
1150 369 "DisplayServerWindows::WndProc@5186-6577@.\platform\windows\display_server_windows.cpp" ".\platform\windows...
1110 354 "RichTextLabel::append_text@5303-6538@.\scene\gui\rich_text_label.cpp"                 ".\scene\gui\rich_t...
```

Figure 1: Top five most complex C++ functions

By applying a more realistic approach, the results could be presented as in Figure 3. In this case, Number of Lines of Code (NLOC) was limited to 150 and CC limited to 30, with both listed in descending order. This to produce a list consisting of the largest values within a reasonable range.

```
NLOC CC Function
---- -- --------
 148 30 "RendererSceneCull::_light_instance_setup_directional_shadow@2135-2354@.\servers\rendering\ren...
 144 30 "ChainIK3DGizmoPlugin::get_joints_mesh@109-269@.\editor\scene\3d\gizmos\chain_ik_3d_gizmo_plug...
 141 30 "AnimationLibraryEditor::_file_popup_selected@155-309@.\editor\animation\animation_library_edi...
 140 30 "AnimationPlayerEditor::_animation_name_edited@613-783@.\editor\animation\animation_player_edi...
 139 30 "CodeSignRequirements::parse_requirements@618-769@.\editor\export\codesign.cpp"
```

Figure 3: Top five functions with NLOC lesser than 150 and CC lesser than 30

- **Did all methods (tools vs. manual count) get the same result?**
  By applying manual calculation, it was discovered that the results, shown in Table 1, were not equal or consistent compared to the ones provided by the tool, see Figure 3. This clearly shows why an automated tool is needed to avoid human error and get a reliable value.

Table 1: Manual CC results

Function	Result 1	Result 2
RendererSceneCull::_light_instance_setup_directional_shadow()	30	23
ChainIK3DGizmoPlugin::get_joints_mesh()	28	25
AnimationLibraryEditor::_file_popup_selected()	24	27
AnimationPlayerEditor::_animation_name_edited()	22	26
CodeSignRequirements::parse_requirements()	27	39

- **Are the results clear?**
  The results provided by the tool, see Figure 1-3, was clear since all the sorting was decided manually as well as the whole .csv file with all relevant functions were provided. This file made it possible to compare the list containing the top five functions to the full list, allowing troubleshooting and verification.

  The manual results on the other hand, visualized in Table 1, were not consistent due to the size and nestled loops and statements making it very difficult to assess manually by hand.

2. **Are the functions just complex, or also long?**
   Since the CC was limited to 30 as well as NLOC limited to 150, the result shows that CC can stay the same while the NLOC value differs. This means that there does not necessarily need to be a correlation between NLOC and CC. In this specific case the length of the function is very similar. But by analysing the five functions with the absolute highest NLOC, using Lizard, it clearly shows that high NLOC does not equal large CC. It is possible to observe that the function with largest NLOC has the second to least CC based on the five functions.

3. **What is the purpose of the functions?**
   By analysing the individual functions in Figure 3 the following could be concluded;
   - _light_instance_setup_directional_shadow() - Handles culling of directional light in 3d scenes. Also stabilizes shadows so their edges do not appear to move. Most of the function is dedicated to various linear algebra graphics transformations.
   - ChainIK3DGizmoPlugin:get_joints_mesh() - The purpose of this function is to visualise IK-chains in 3D-gizmos. The functions is specifically used by the editor to visualize a inverse kinematic (IK) gizmo on a Skeleton3D. It draws lines and rotation limitations within the joints of the chain.
   - _file_popup_selected() - This function is a GUI callback that handles all file-related popup menu actions for both animation libraries and individual

animations in the Animation Library editor panel. When a user right-clicks a library or animation entry, this function dispatches to one of eight behaviors based on the menu item selected: saving a library/animation to its existing path, saving to a new path ("Save As"), making a library/animation unique (deep-copying so it is no longer shared), or opening it in the Inspector. The high CC is a direct consequence of the eight mutually-exclusive case branches plus nested path-validation logic that checks whether a resource is embedded in a scene file, standalone on disk, or imported (read-only).

- _animation_name_edited() - Handles various events that happen when editing the name of an animation. Stops the animation if it's playing, reports if the name is valid, hides the naming dialog, prevents duplicate names, puts the action in the editor undo queue with various cases depending on how it was created.
- parse_requirements() - Due to the lack of documentation, the purpose of this function is largely inscrutable. It reads a “requirement header” and creates a list of strings based on it. It also returns various errors for invalid headers.

4. **Are exceptions taken into account in the given measurements?**
   C++ exceptions are not used in Godot’s code, outside of 3rd party dependencies, so there was nothing to take into account.

5. **Is the documentation clear w.r.t. all the possible outcomes?**
   No, none of these functions have comments describing their overall function in the code and not in the online documentation either.

Refactoring

_file_popup_selected()

The high CC of 30 in _file_popup_selected() is largely accidental complexity rather than necessary complexity. Each of the eight switch cases represents a fully independent behavior, yet they are all combined into a single function. The nested path-validation logic (checking whether a resource is embedded in a scene file, a standalone resource file, or an imported read-only file) also appears identically for both the library and animation save cases, meaning there is duplicated logic inside an already large function.

Plan for refactoring complex code:

- _save_library() — handles FILE_MENU_SAVE_LIBRARY and its fallthrough
- _save_animation() — handles FILE_MENU_SAVE_ANIMATION and its fallthrough
- _make_library_unique() — handles FILE_MENU_MAKE_LIBRARY_UNIQUE
- _make_animation_unique() handles FILE_MENU_MAKE_ANIMATION_UNIQUE
- _edit_library() — handles FILE_MENU_EDIT_LIBRARY
- _edit_animation() — handles FILE_MENU_EDIT_ANIMATION
- _get_save_error(const String &p_path) — shared helper returning an error string if the resource is not allowed to be saved, used by both _save_library() and _save_animation()

Estimated impact of refactoring (lower CC, but other drawbacks?).
After this refactoring, the dispatcher becomes a simple 10 line switch with CC=9. The largest extracted method (_save_library) would have CC ≈ 5. No individual unit would exceed CC5, and the total CC across all methods drops from 30 to approximately 27. But critically, each piece is now independently readable and testable. The only drawback is six additional private methods on the AnimationLibraryEditor class, slightly increasing its interface size.

Link from PR: https://github.com/Soffan-Group-6/godot/pull/9

String::append_utf16

The if block handling byte order markers could be separated out into its own function, reducing CC by about 5. Similarly, the for loop checking for invalid surrogate characters and counting string length could be split into its own function reducing CC by about 6. These changes would lower the functions CC from 27 to about 16.

Geometry3D:get_closest_points_between_segments

The reason behind high cyclomatic complexity for this function is due to its multi-function, combining input validation, branching on modes/types, special case handling, etc. This leads to a number of nested if-then-else blocks and long switch statements. It is partly necessary for the function to be highly complex.
However, it is possible to refactor: Extract smaller helper functions for subtasks so each function has fewer branches (for example, `validate_input()`); Replace deep nested `if` with guar clauses (early returns) to simplify the structure; Replace long if-then-else chains with clearer construct such as `switch` statements; Isolate special-case and error handling into helper functions
After all, by splitting the function into smaller functions, the CC can be reduced to about 5 if branches in helpers are not calculated, but if branches inside the helpers are calculated, CC value will largely remains the same

ChainIK3DGizmoPlugin::get_joints_mesh()

The function contains a lot of nestled loops and if-statements that, for the purpose of making the code clearer, could be moved into other partial functions for the intended purpose. The if-statements are checking certain requirements that has to be passed for certain lines to be drawn, transformations and more to finally conclude the meshes. What happens within these requirements could become much clearer for the coder by adding a function that performs the tasks related to the respective requirement. This especially for the “parent” if-statements in the second nestled for-loop. The special case of the extended ending joint contains a very large chunk of code that definitely could be parted into its own function with a describing name telling the coder what the purpose of this code is. In addition the complexity could possibly be reduced by reducing some of the nestled if-statements into one by combining the passing criteria into a function or object. It may also be possible to use switch in some parts of the function.

String::simplify_path()

The function is quite long and handles several distinct tasks connected to simplifying a given path. The function could therefore be broken down into smaller functions where each has single responsibility. For example, the block that removes “.” and “..” directories in the given path could become its own function, creating a function with cyclomatic complexity of around 5. Similarly, the reconstruction of the simplified path to be returned could be a separate function. By doing refactoring this way, each function becomes smaller and also easier to test.

Coverage

Table 2: Functions with LOC and CCN according to lizard:

Function	LOC	CCN
String::simplify_path()	74	24
Geometry3D::get_closest_distance_between_segments()	72	20
String::append_utf16()	101	27
ChainIK3DGizmoPlugin::get_joints_mesh()	144	30
_file_popup_selected()	141	30

Tools

We used Gcov for code instrumentation. It was relatively easy to integrate in the build environment since there already was an option to enable it in the build process. The way to enable it wasn't well documented, and involved changing a line in a random python file. After that it build correctly with instrumentation. Figuring out how to get data out of gcov took a moment as the source code needed to be moved to the build directory for gcov to detect it and create annotated source files.

Your own coverage tool

Manual instrumentation branch:
https://github.com/Soffan-Group-6/godot/tree/manual-branch-coverage

What kinds of constructs does your tool support, and how accurate is its output?

The code coverage tool implemented for all functions supports constructs in terms of if-statements and for-loops.

_file_popup_selected()

Branch:
https://github.com/Soffan-Group-6/godot/tree/feat/7-coverage-instrumentation-file-popup-selected

The tool uses a static boolean array of size 30 with atexit() to print branch hit/miss results to stderr on program exit. It covers all if/else outcomes and switch/case entries (30 branches total). Compound boolean expressions using && and || are treated as single branch points since splitting them would require rewriting the conditions rather than inserting a flag. No ternary operators or C++ exceptions appear in the function. Because the function is editor-only (depends on EditorNode, get_tree(), and file_dialog), the headless test runner never reaches it, so coverage is verified during live editor sessions.

Evaluation

get_closest_distance_between_segments()

For instance, Figure 4 shows the branch coverage by manual instrumentation

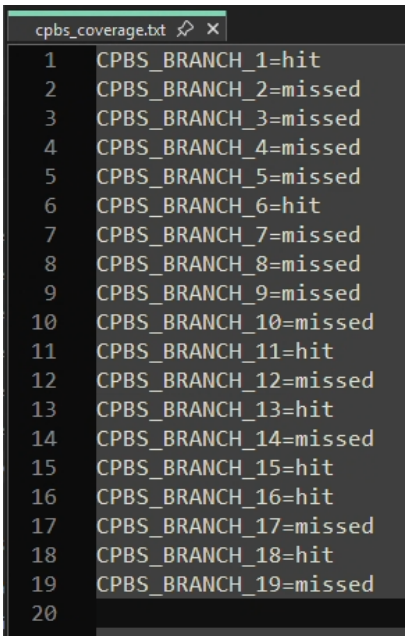

Figure 4: Initial branch coverage

and Figure 5 shows the branch coverage for the file core/math/geometry_3d.cpp

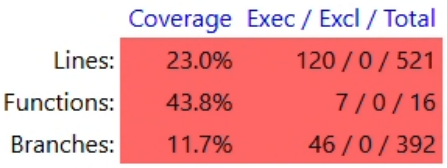

Figure 5: Improved branch coverage

By counting the branches coverage from the output (that produces Figure 4), the branch coverage of the function is measured to be 9/18 (50%).

It is clear that the manual and automated tool is not consistent.

How detailed is your coverage measurement?

Since the tool requires manual placement of “hitpoints” some coverage points might be missed by mistake. Apart from that the tool measures both for-loops and if-statements. Ternary operations were excluded since it would have required modification of the code, meaning it was not possible to just insert a flag.

What are the limitations of your own tool?

The tool is, as specified before, limited to manual implementation of flags. At the same time it is possible to apply the functionality to any function by adding it in the branch_coverage() file and manually place hitpoints in the specified function. From there the limitations is in the hands of the developer in terms of where flags are implemented.

Are the results of your tool consistent with existing coverage tools?

ChainIK3DGizmoPlugin::get_joints_mesh()

The results produced by the manually implemented tool can be seen in Figure 6. In addition the result produced by an existing code coverage tool is shown in Figure 7. It is clearly visualized that no unit tests had been previously implemented for the function, therefore, the manual result produced zeros for all the tracked branches. Verified by using an exiting tool for code coverage it could be concluded that the results were the same, meaning no coverage implemented in terms of tests.

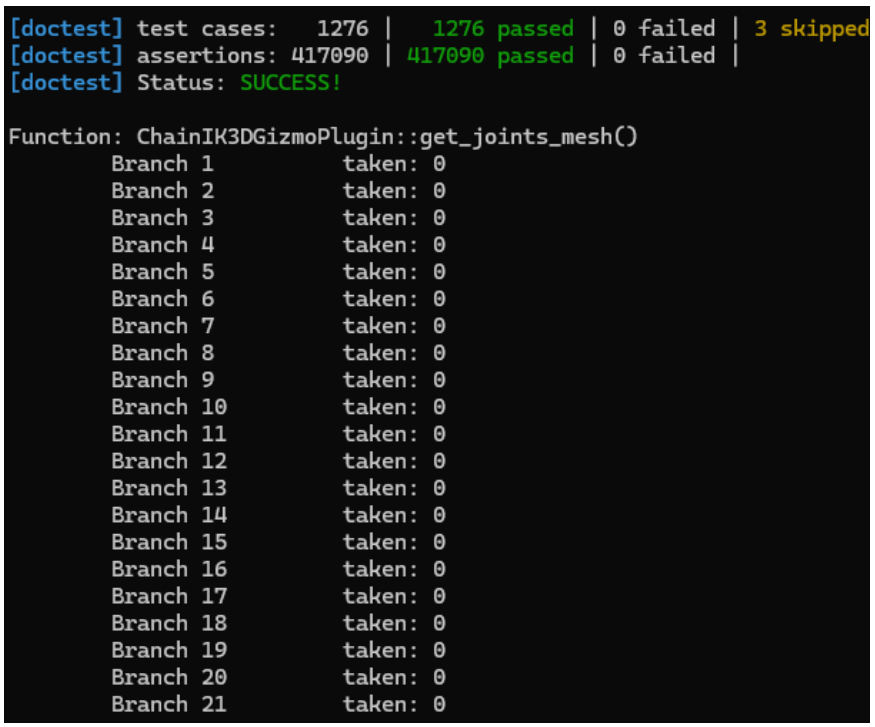

Figure 6: Manual code coverage test result for "ChainIK3DGizmoPlugin::*get_joints_mesh()*"

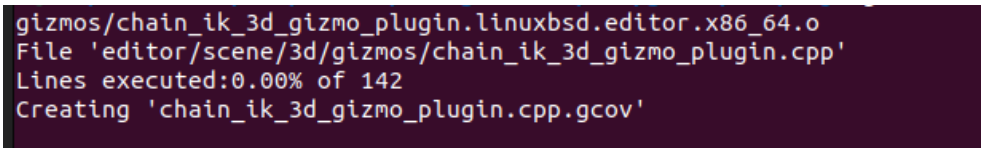

Figure 7: Code coverage test result for "ChainIK3DGizmoPlugin::*get_joints_mesh()*"

String::append_utf16()

The same instrumentation method was used as in ChainIK3DGizmoPlugin::*get_joints_mesh()*.

Gcov's branch coverage results generally match the manual instrumentation ones. Gcov indicates 8 code blocks are never reached while according to manual instrumentation it was 7.

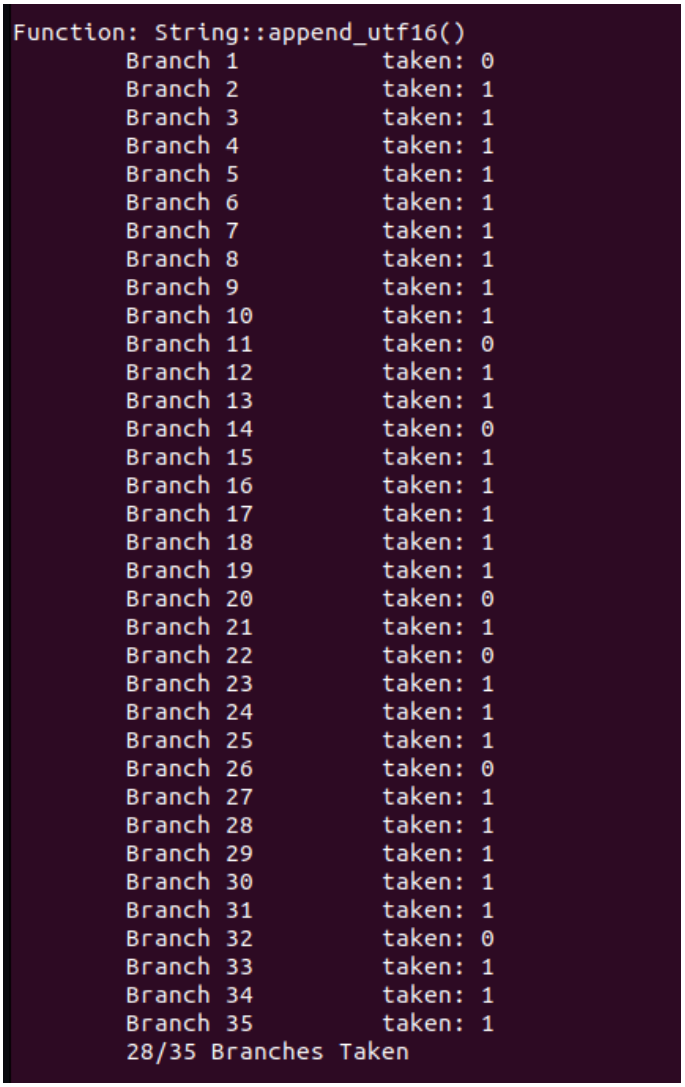

Figure 8: Manual code coverage

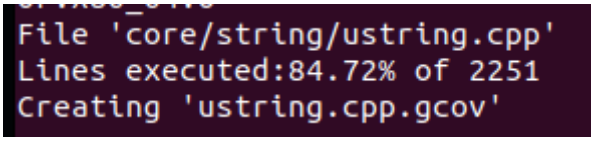

Figure 9: Overall gcov data for file

simplify_path()

Initially, the manually implemented coverage tool indicated that 6 branches out of 29 were not executed in the simplify_path() function.

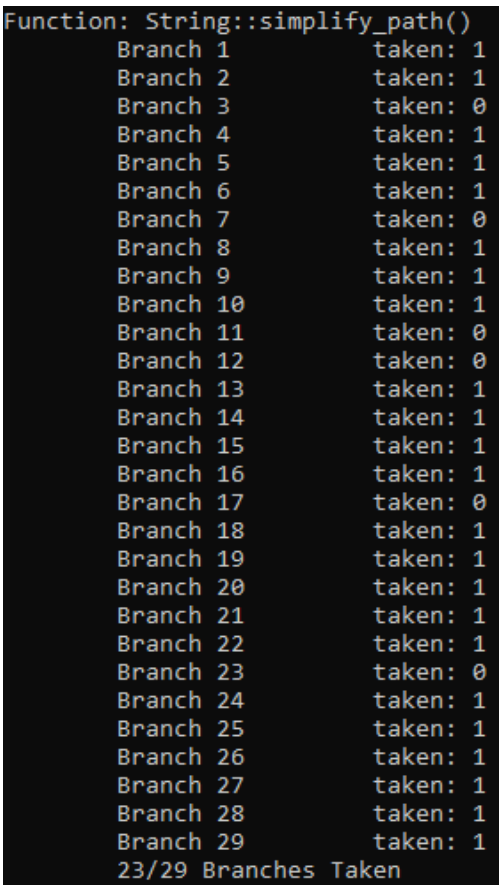

Figure 10: Manual code coverage on simplify_path()

Coverage improvement

The following branch implements code coverage improvement through unit tests: github.com/Soffan-Group-6/godot/tree/code-coverage-improvement

Coverage improvement for String::simplify_path()

After creating and running 4 additional unit tests forcing execution of 4 branches, the number of uncovered branches decreased to 2, demonstrating that the new tests effectively improved branch coverage.

Initial branch coverage:

```
Function: String::simplify_path()
        Branch 1          taken: 1
        Branch 2          taken: 1
        Branch 3          taken: 0
        Branch 4          taken: 1
        Branch 5          taken: 1
        Branch 6          taken: 1
        Branch 7          taken: 0
        Branch 8          taken: 1
        Branch 9          taken: 1
        Branch 10         taken: 1
        Branch 11         taken: 0
        Branch 12         taken: 0
        Branch 13         taken: 1
        Branch 14         taken: 1
        Branch 15         taken: 1
        Branch 16         taken: 1
        Branch 17         taken: 0
        Branch 18         taken: 1
        Branch 19         taken: 1
        Branch 20         taken: 1
        Branch 21         taken: 1
        Branch 22         taken: 1
        Branch 23         taken: 0
        Branch 24         taken: 1
        Branch 25         taken: 1
        Branch 26         taken: 1
        Branch 27         taken: 1
        Branch 28         taken: 1
        Branch 29         taken: 1
        23/29 Branches Taken
```

After added 4 tests

```
Function: String::simplify_path()
        Branch 1          taken: 1
        Branch 2          taken: 1
        Branch 3          taken: 0
        Branch 4          taken: 1
        Branch 5          taken: 1
        Branch 6          taken: 1
        Branch 7          taken: 1
        Branch 8          taken: 1
        Branch 9          taken: 1
        Branch 10         taken: 1
        Branch 11         taken: 1
        Branch 12         taken: 1
        Branch 13         taken: 1
        Branch 14         taken: 1
        Branch 15         taken: 1
        Branch 16         taken: 1
        Branch 17         taken: 1
        Branch 18         taken: 1
        Branch 19         taken: 1
        Branch 20         taken: 1
        Branch 21         taken: 1
        Branch 22         taken: 1
        Branch 23         taken: 0
        Branch 24         taken: 1
        Branch 25         taken: 1
        Branch 26         taken: 1
        Branch 27         taken: 1
        Branch 28         taken: 1
        Branch 29         taken: 1
        27/29 Branches Taken
```

Figure 11: Branch coverage before and after

Coverage improvement for String::append_utf16()

3 tests were added, for nullptr data, empty strings and invalid character sequences of surrogate characters. This increased branch coverage to 100%.

Initial branch coverage: After 3 added tests:

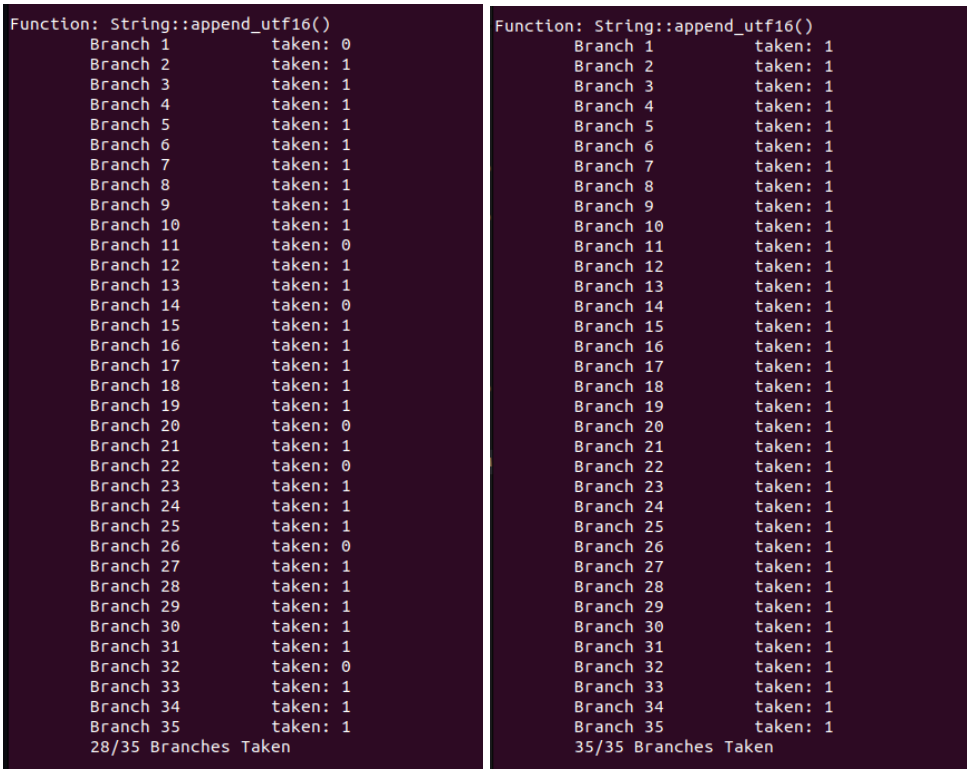

Figure 12: Branch coverage before and after

Geometry3D::get_closests_points_between_segments()

The below shows the general branch coverage on the file geometry_3d.cpp, but the significance can be seen with the difference between them. (There is a discrepancy about some data since some adjustment of newly added functions are being removed when conducting initial and improved branch coverage)

	Coverage	Exec / Excl / Total
Lines:	23.7%	128 / 0 / 541
Functions:	47.1%	8 / 0 / 17
Branches:	11.7%	46 / 0 / 392

Figure 13: Initial branch coverage

	Coverage	Exec / Excl / Total
Lines:	25.3%	132 / 0 / 521
Functions:	43.8%	7 / 0 / 16
Branches:	14.8%	58 / 0 / 392

Figure 14: Improved branch coverage

ChainIK3DGizmoPlugin::get_joints_mesh()

By creating test cases on a mocked version of ChainIK3DGizmoPlugin::get_joints_mesh() the result became as shown in Figure 15. Compared to the initial 0% caused by the function not being unit tested at all, the result was a large improvement. Due to the function being a gizmo and requiring a real functioning editor environment because of the objects used within the function, the only way to perform unit tests was by mocking stripped versions of all objects requiring a real time editor environment running. By mocking them the function could be successfully unit tested to check code coverage depending on what parameters are delivered to the function.

```
File 'tests/scene/mocks/mock_chain_ik_3d_gizmo_plugin.cpp'
Lines executed:79.37% of 63
Creating 'mock_chain_ik_3d_gizmo_plugin.cpp.gcov'
```

Figure 15: Improved code coverage results of ChainIK3DGizmoPlugin::get_joints_mesh() through unit tests

Self-assessment: Way of working

As a group, we assess our way of working to be in the “in use” state according to the Essence standard. Throughout the assignment, we agreed upon applying principles and practises regarding performing complexity measurements, manual coverage, and coverage increasing unit-tests. For example, we divided responsibilities for measuring cyclomatic complexity and collaboratively implemented branch coverage tracking for five complex functions. Initially however, the implementations of manual branch coverage differed somewhat between group members which eventually required the different implementations to be refactored to follow a common structure.

One challenge we faced as a team was that several complex functions that were chosen initially were connected to the editor interface or other system components, making them difficult to test in isolation. This created some uncertainty in the workflow when selecting which functions to instrument and how to measure branch coverage. This resulted in some discussion and coordination among team members to get everyone on board, and in the end we came to the conclusion that we had to make sure that the functions we chose for part 2 of the assignment would be testable. This also explains why some of the functions used in part 1 and 2 differed.

According to the essence checklist, our way-of-working is “in use”. The principles and practices we agreed on are actively applied by the team, and they support communication, collaboration, and progress on tasks. Some practices are still developing and not yet applied in exactly the same way by all members. Overall the team is able to complete tasks effectively, share knowledge, and coordinate work without problem.

In order to reach the next state “in place”, greater unity within the group would be required to ensure that all members are aware of how to implement functionality in a way that aligns with the common structure being used. The criterion that all team members should have access to the practices and tools required to perform their work does also need to become satisfied, as gcov for e.g. could not be run by all group members.

Overall experience

The project is a bit challenging to work on since the selected project is quite large relative to the available time for the assignment. The main takeaway is learning to go from raw coverage numbers to targeted requirements and tests for an actual C++ function in a large codebase. During the work, we spent a lot of time discovering tools that are available for us to do the analysis, branch coverage, especially, and how we can get a reliable pipeline to work on.

There are quite a lot of takeaways in this assignment, except for those mentioned above, it is important for us to learn how to read existing algorithms, and how to analyze the numbers to more concrete ideas; For instance, interpreting "X % covered" into a map of concrete untested behaviors by existing test suite, and extends to building more test cases to cover corresponding untested lines or branches of the algorithms. Another knowledge that is valuable during the work is toolchain awareness, especially for Windows, which has multiple different tools that could lead to different format of report, but doing analysis; Moreover, it is pretty complicated and somewhat annoying to work on it on Windows, and requires the help from team members who are on Linux.

Speaking of honorable mention, it is eye-opening how subtle numeric edge cases can hide in seemingly simple functions, and how coverage tools can help to reveal locations the tests never touch. In summary, the project is interesting to play with, in addition to large amount of hands-on experience on working on some existing projects available to the public.

Statement of contribution

Erik Olsson (erik-ol):

- Initial verification that Godot is feasible to use
- Manual CCN count of 5 complex functions
- Manual instrumentation of String::append_utf16()
- Additional tests for String::append_utf16()

Chow Pun Chun (MrNoodlez-1227):

- Manual instrumentation of Geometry3D::get_closests_points_between_segments()
- Additional tests for Geometry3D::get_closests_points_between_segments()

Albin Blomqvist (zzimbaa):

- Manual instrumentation of String::simplify_path()
- Additional tests for String::simplify_path()
- Essence documentation

Josefine Nyholm (joss2002):

- Manual CCN count of 5 complex functions.
- Using automated tool to find 5 functions with high CCN
- Manual instrumentation of ChainIK3DGizmoPlugin::get_joints_mesh()
- Coverage improvement/additional tests for ChainIK3DGizmoPlugin::get_joints_mesh()

Avid “HotFazz” Fayaz

- Manual CCN count of AnimationLibraryEditor::_file_popup_selected()
- Manual instrumentation of AnimationLibraryEditor::_file_popup_selected()
- Additional testing for AnimationLibraryEditor::_file_popup_selected()
- Refactoring plan for AnimationLibraryEditor::_file_popup_selected()

References

[1] *Godot Engine Documentation,* “Compiling for Windows”, *docs.godotengine.org,* Accessed: Feb. 17, 2026. [Online]. Available: https://docs.godotengine.org/en/latest/engine_details/development/compiling/compiling_for_windows.html